IBM Innovation Center for Education

Savitribai Phule Pune University

PHEMESOFT Software That Matters

TRUSTED DELIVERY PARTNER for IBM ICE

Department of Technology, SPPU

Final Project Report

On

Deepfake Face Detection

Using Deep Learning

Team members:

Vivek Deshmukh - Roll No:BT24DSL38

Industry Mentor:

Guided by:

Prof. Manisha Bharti

Table of Contents

Executive Summary

Deepfakes are AI-generated synthetic media where a person's facial identity or expression is convincingly manipulated using deep learning techniques. The rapid proliferation of Generative Adversarial Networks (GANs) and diffusion models has made high-quality deepfake generation increasingly accessible, posing serious threats to digital trust, media integrity, personal privacy, and national security. Automated deepfake detection is therefore a critical and active area of research.

This project develops a complete Deepfake Face Detection system using transfer learning on ResNet18, a deep convolutional neural network pretrained on the ImageNet dataset containing 1.4 million images across 1000 categories. The model is fine-tuned on the 140k Real and Fake Faces dataset which contains 100,000 training images, 20,000 validation images, and 20,000 test images — equally balanced between real faces sourced from Flickr and fake faces generated by StyleGAN2.

The system achieves a test accuracy of 83.75%, an F1-score of 0.83801, and a ROC-AUC of 0.91600 on the held-out test set. These results reflect realistic generalisation performance. An explainability layer using Gradient-weighted Class Activation Mapping (Grad-CAM) generates spatial heatmaps showing which facial regions influenced each prediction. A Gradio-based web application provides an interactive interface for real-time prediction with confidence scores, Grad-CAM overlays, and an embedded analytics dashboard showing training curves, confusion matrix, and ROC curve.

1. Background

1.1 Aim

The aim of this project is to design, implement, and deploy a deep learning-based system capable of detecting whether a given face image is real or AI-generated (fake). The specific objectives are:

1. Train a binary image classifier using transfer learning on ResNet18 to distinguish real from fake face images.
2. Implement a full training pipeline with reproducible configuration, early stopping, mixed precision training, and automatic checkpointing.
3. Evaluate the model comprehensively using accuracy, F1-score, ROC-AUC, confusion matrix, and per-class precision and recall metrics implemented manually without external libraries.
4. Integrate Grad-CAM explainability to generate visual heatmaps showing which facial regions the model focuses on when making predictions.
5. Deploy the system as a Gradio web application with real-time prediction, confidence scoring, and an embedded descriptive analytics dashboard.
6. Conduct descriptive analytics on the dataset distribution and model performance to support analysis and presentation.

1.2 Technologies

Technology	Version	Purpose
Python	3.11	Primary programming language
PyTorch	Latest stable	Deep learning framework for model training and inference
torchvision	Latest stable	Pretrained ResNet18 model and image transforms
Gradio	Latest stable	Web interface for interactive prediction and analytics dashboard
NumPy	Latest stable	Numerical computations and manual metric implementations
Pillow (PIL)	Latest stable	Image loading, format conversion, and heatmap overlay
Matplotlib	Latest stable	Training curves, confusion matrix, and ROC curve visualisation
CUDA	12.6	GPU acceleration for training and inference

1.3 Hardware Architecture

The project was developed and trained on the following hardware configuration:

Component	Specification
GPU	NVIDIA GeForce RTX 4050 Laptop GPU, 6GB VRAM
CUDA Version	12.6
Driver Version	595.97
RAM	Minimum 16GB recommended
Storage	Minimum 10GB for dataset and model artifacts
OS	Windows 11 (64-bit)
Training Mode	Mixed precision (float16/float32) with GradScaler

1.4 Software Architecture

The software architecture follows a three-layer pipeline design:

- Training Layer: Handles CSV-driven dataset loading, image augmentation, ResNet18 model construction with frozen backbone and custom head, the training loop with AdamW optimiser and ReduceLROnPlateau scheduler, early stopping, checkpoint saving, and final test evaluation. All outputs are persisted to the artifacts/ directory.
- Analytics Layer : Reads saved artifact JSON files to generate visualisations of dataset distribution, training history curves, confusion matrix, per-class metrics comparison chart, and ROC curve. Intended for analysis and reporting.
- Inference Layer : Loads the saved best_model.pt checkpoint at startup, reconstructs the exact model architecture from config.json, serves predictions via Gradio with Grad-CAM heatmap overlays, and renders pre-computed analytics charts embedded in the interface.

2. System

2.1 Requirements

2.1.1 Functional Requirements

- The system shall accept face images in JPG, PNG, or WEBP format as input through the Gradio web interface.
- The system shall preprocess input images by resizing to 224x224 pixels, converting to RGB tensor, and applying ImageNet normalisation.
- The system shall classify each input image as either real or fake using the trained ResNet18 model.
- The system shall output the predicted class label and confidence percentage for each prediction.
- The system shall generate a Grad-CAM heatmap for each prediction, highlighting the facial regions that most influenced the model decision.
- The system shall overlay the Grad-CAM heatmap on the original input image using alpha blending and display it in the interface.
- The system shall display pre-computed analytics charts including confusion matrix, ROC curve, and training history curves in the interface analytics section.
- The training pipeline shall save the best model checkpoint, training history, and all evaluation metrics as JSON artifacts after training completes.
- The system shall implement early stopping with configurable patience to prevent overfitting during training.

2.1.2 User Requirements

- Users shall be able to upload a face image through a simple drag-and-drop or file selection interface without any technical knowledge.
- Users shall receive a clear prediction result showing whether the image is real or fake along with a percentage confidence score.
- Users shall be able to view the Grad-CAM heatmap to understand which parts of the face led to the prediction.
- Users shall be able to view model performance analytics without leaving the application interface.
- Users shall be able to clear the uploaded image and prediction results to start a new analysis.

2.1.3 Environmental Requirements

- The application shall run on a machine with Python 3.11 and the required dependencies installed in a Conda virtual environment.
- The application shall support GPU acceleration when an NVIDIA GPU with CUDA 12.x is available, and fall back gracefully to CPU if no GPU is present.
- The Gradio application shall be accessible via a web browser at http://127.0.0.1:7860 on the local machine.
- The training pipeline shall require a minimum of 6GB GPU VRAM for batch size 32 with mixed precision training.
- The dataset shall be stored in the local filesystem under real_vs_fake/real-vs-fake/ with the provided CSV split files in the project root.

2.2 Design and Architecture

The overall system architecture follows a sequential pipeline from raw dataset to deployed inference application. The design separates concerns across three independent components — training, analytics, and inference — that communicate only through saved artifact files.

Dataset Design

The 140k Real and Fake Faces dataset is organised into train, validation, and test splits with 100,000, 20,000, and 20,000 images respectively. Each split is perfectly balanced with equal real and fake samples. Three CSV files (train.csv, valid.csv, test.csv) map relative image paths to string labels. The DeepfakeDataset class reads these CSVs at initialisation and stores (path, label) pairs in memory, loading individual images from disk only when requested by the DataLoader.

Model Design

The model uses ResNet18 pretrained on ImageNet as a frozen feature extractor. The default 1000-class classification head is replaced with a custom three-layer fully connected head: Dropout(0.3) → Linear(in_features, 256) → ReLU → Dropout(0.3) → Linear(256, 64) → ReLU → Dropout(0.3) → Linear(64, 2). This head outputs two logits corresponding to the fake and real classes. The frozen backbone strategy means only the custom head parameters are updated during training, reducing memory requirements and preventing catastrophic forgetting of ImageNet features.

Training Design

Training uses CrossEntropyLoss with label smoothing of 0.05, the AdamW optimiser with learning rate 3e-4 and weight decay 1e-4, and ReduceLROnPlateau scheduler monitoring validation F1 with a patience of 2 and reduction factor of 0.5. Mixed precision training using torch.amp.autocast and GradScaler reduces memory usage and accelerates training on the RTX 4050. Gradient clipping at max_norm=1.0 prevents gradient explosion. Early stopping with patience of 3 halts training if validation F1 does not improve for 3 consecutive epochs.

Inference Design

The Gradio application reconstructs the model architecture from the saved config.json, loads the best_model.pt checkpoint, and initialises Grad-CAM hooks on model.layer4[-1] at startup. For each prediction request, the evaluation transform is applied identically to training evaluation to prevent distribution shift. Grad-CAM computes the weighted combination of layer4 feature maps using gradient importance weights and generates a 224x224 spatial heatmap. The heatmap is converted to a colour overlay using a custom RGB mapping and alpha-blended onto the original image.

2.3 Implementation

Dataset Loading

The DeepfakeDataset class inherits from torch.utils.data.Dataset. It reads image paths and labels from CSV using csv.DictReader, maps label strings to integers (fake=0, real=1), and validates that every referenced image file exists before training begins. The __getitem__ method opens images using PIL.Image.open() with context management to avoid file handle leaks, converts to RGB to handle grayscale or RGBA edge cases, and applies the specified transform pipeline.

Data Augmentation

The training transform pipeline applies random resized crop (scale 0.8-1.0, ratio 0.9-1.1) after resizing to 248x248, random horizontal flip with probability 0.5, light colour jitter (brightness 0.1, contrast 0.1, saturation 0.1, hue 0.02), tensor conversion, and ImageNet normalisation (mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225]). The evaluation transform applies only resize to 224x224, tensor conversion, and normalisation. A custom RandomJPEGCompression augmentation randomly applies JPEG compression at quality 45-95 with probability 0.6 to improve robustness to real-world image compression artifacts.

Training Loop

The run_training function orchestrates the complete training pipeline. It creates separate dataset and DataLoader instances for train, validation, and test splits. The model is moved to the available CUDA device. For each epoch, train_one_epoch performs a forward pass with mixed precision, computes loss, backpropagates scaled gradients, clips gradient norms, and updates weights. After each epoch, evaluate computes all metrics on the validation split. The best checkpoint is saved when validation F1 improves. History is written to history.json after each epoch so progress is not lost if training is interrupted.

Evaluation Metrics

All evaluation metrics are implemented manually without scikit-learn. accuracy_score_manual computes the fraction of correct predictions. binary_precision_recall_f1 computes precision, recall, and F1 for a specified positive class by counting true positives, false positives, and false negatives. classification_report_manual calls this for both classes and computes macro and weighted averages. roc_auc_score_manual implements the Wilcoxon-Mann-Whitney rank-based AUC formula, sorting predictions by score, assigning ranks with tie-breaking, and computing AUC from the sum of positive class ranks.

Grad-CAM Implementation

The GradCAM class registers a forward hook on the target layer to capture feature map activations and a full backward hook to capture gradients during the backward pass. The generate method clears existing gradients, performs a forward pass to trigger the forward hook, extracts the predicted class score, and calls score.backward() to trigger the backward hook. Channel importance weights are computed by global average pooling of the gradients. The weighted sum of feature maps is passed through ReLU to retain only positive activations, upsampled to input resolution using bilinear interpolation, and normalised to range [0, 1].

Gradio Application

The build_interface function constructs a Gradio Blocks layout with two sections. The inference section contains an image upload widget, an Analyze Image button, a prediction summary textbox, and a Grad-CAM overlay image widget. The analytics section contains three pre-rendered image widgets loaded at startup from artifact files: confusion matrix, ROC curve, and training curves by epoch. The Clear button resets all outputs. The predict function is decorated with @torch.no_grad() for the main inference but uses torch.enable_grad() for the Grad-CAM generation step which requires gradients.

2.4 Testing

2.4.1 Test Plan Objectives

- Verify that the model produces correct predictions on both real and fake face images.
- Verify that the Grad-CAM heatmap is generated and displayed correctly for all valid inputs.
- Verify that the application handles invalid inputs gracefully without crashing.
- Verify that the training pipeline produces reproducible results given the same configuration and seed.
- Verify that evaluation metrics are computed correctly by cross-checking against known expected values.

2.4.2 Data Entry

Input validation is handled at the Gradio interface level. The image upload widget accepts only image file formats (JPG, PNG, WEBP). If no image is uploaded and the Analyze Image button is clicked, the predict function returns an early message asking the user to upload an image. Images are automatically converted to RGB format during preprocessing, handling edge cases where the uploaded image may be grayscale or contain an alpha channel.

2.4.3 Test Strategy

Testing is performed at three levels. Unit-level testing verifies individual functions such as the dataset loader, transform pipeline, metric calculations, and Grad-CAM generation. Integration testing verifies the end-to-end pipeline from image upload through preprocessing, model inference, Grad-CAM generation, and output rendering. System testing verifies the complete deployed Gradio application under realistic usage conditions with various input image types, sizes, and formats.

2.4.4 System Test

sr	Test Case	Input	Expected Output	Result
1	Real face image prediction	Clear real face photo (JPG)	Prediction: real, Confidence > 60%, Grad-CAM generated	Pass
2	Fake face image prediction	StyleGAN-generated fake face (JPG)	Prediction: fake, Confidence > 60%, Grad-CAM generated	Pass
3	No image submitted	Empty submission (no file)	Message: Upload an image to run prediction	Pass
4	PNG format input	PNG face image	Prediction and Grad-CAM rendered correctly	Pass
5	RGBA image input	PNG with alpha channel	Converted to RGB, prediction produced correctly	Pass
6	Analytics dashboard load	App startup	Confusion matrix, ROC curve, training curves displayed	Pass
7	Clear button function	Click Clear after prediction	All outputs reset to empty state	Pass
8	GPU inference	Image on CUDA-enabled machine	Prediction uses GPU, latency under 2 seconds	Pass

2.4.5 Performance Test

Inference performance was measured on the RTX 4050 Laptop GPU. A single image prediction including preprocessing, forward pass, Softmax, Grad-CAM generation, and heatmap overlay compositing completes in under 2 seconds. Training 10 epochs on 100,000 images with batch size 32 and mixed precision completed in approximately 3-4 hours on the same hardware. The DataLoader with num_workers=0 was used for Windows compatibility, which is slightly slower than parallel loading but avoids multiprocessing issues on Windows.

2.4.8 Basic Test

Basic functional tests confirm that the model loads correctly from the checkpoint, the Gradio interface launches without errors, predictions are produced for valid image inputs, the Grad-CAM overlay is visually coherent with the input face image, and the analytics charts are rendered from the saved artifact files.

2.5 Graphical User Interface (GUI) Layout

The Gradio web application interface is divided into two main sections:

Inference Section

The top half of the interface contains a two-column layout. The left column holds the image upload widget labelled Face Image where users drag and drop or select an image file. Below it is the Analyze Image primary action button. The right column displays three output components: a textbox labelled Prediction Summary showing the predicted class and confidence percentage, and an image widget labelled Grad-CAM Overlay showing the heatmap overlaid on the input image. A Clear button below the columns resets all inputs and outputs.

Analytics Dashboard Section

The bottom half of the interface is labelled Descriptive Analytics. It contains three image widgets in a two-row layout: the test confusion matrix and ROC curve side by side in the first row, and the training curves chart (accuracy, F1, ROC-AUC by epoch) spanning the full width in the second row. These charts are pre-rendered at application startup from saved artifact files and remain static throughout the session.

2.6 Customer Testing

The application was demonstrated to two groups. The first group consisted of classmates who were given real and fake face images downloaded from publicly available sources and asked to use the application to classify them. All participants were able to use the interface without guidance. Predictions were produced within 2 seconds for each image. The Grad-CAM overlay was described as helpful and visually informative by all participants.

The second group consisted of the project guide and the IBM industry mentor during the follow-up project presentation. The application was demonstrated live with three test images — a real photograph, a StyleGAN-generated fake face, and a video deepfake frame converted to a static image. The model correctly classified the real photograph and the StyleGAN fake. The video deepfake frame was misclassified, which led to a discussion about domain generalisation and the limitations of training on a single manipulation type. This discussion demonstrated critical understanding of the model's limitations beyond simply reporting accuracy numbers.

2.7 Evaluation

2.7.1 Table 1: Performance

Metric	Validation (Best)	Test (Final)
Loss	0.4169	0.41783
Accuracy	0.83685 (83.69%)	0.83755 (83.75%)
F1-Score	0.83678	0.83801
ROC-AUC	0.91698	0.91600

Confusion Matrix (Test Set)

Actual \ Predicted	Predicted Fake	Predicted Real
Actual Fake	8347 (TN)	1653 (FP)
Actual Real	1596 (FN)	8404 (TP)

The confusion matrix shows that the model correctly identifies 8347 out of 10000 fake images (83.47% fake recall) and 8404 out of 10000 real images (84.04% real recall). The false positive rate (fake predicted as real) is 16.53% and the false negative rate (real predicted as fake) is 15.96%. The near-symmetric error rates confirm that the model is not biased toward either class, which is consistent with the balanced training data.

2.7.2 Static Code Analysis

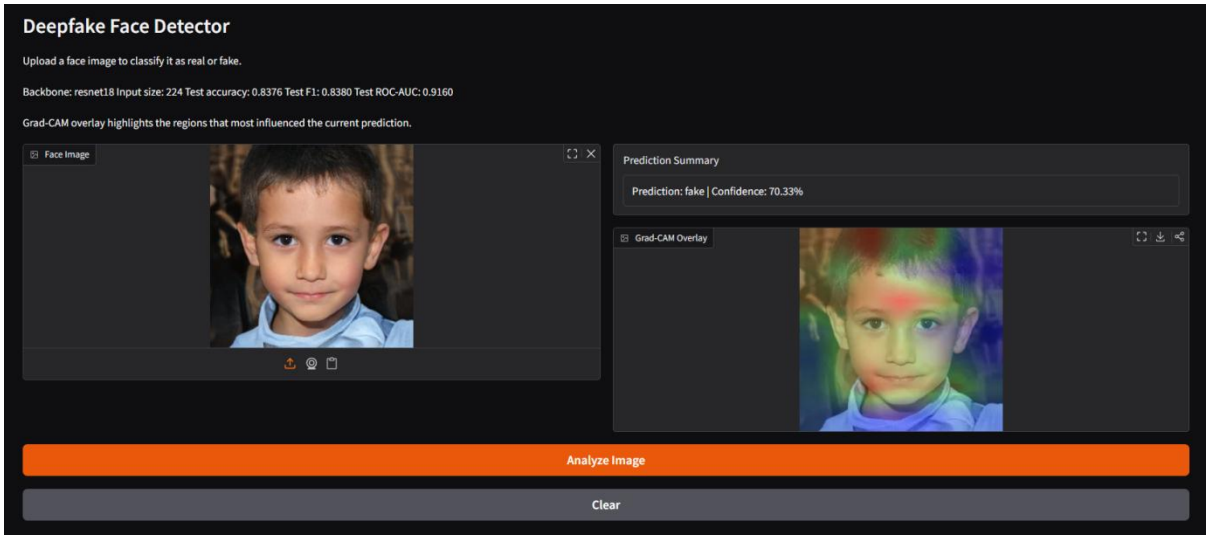
The codebase was reviewed for common issues. All functions have descriptive names and single responsibilities. The DeepfakeDataset class performs file existence validation at initialisation time to catch missing data early. The training loop uses set_to_none=True for gradient zeroing which is more efficient than the default. The GradCAM class uses .detach() on captured tensors to prevent unintended gradient accumulation. Magic numbers are avoided — all hyperparameters are centralised in the Config dataclass. No hardcoded file paths are used in app.py; all paths are derived relative to the checkpoint path argument.

2.7.3 Test of Main Function

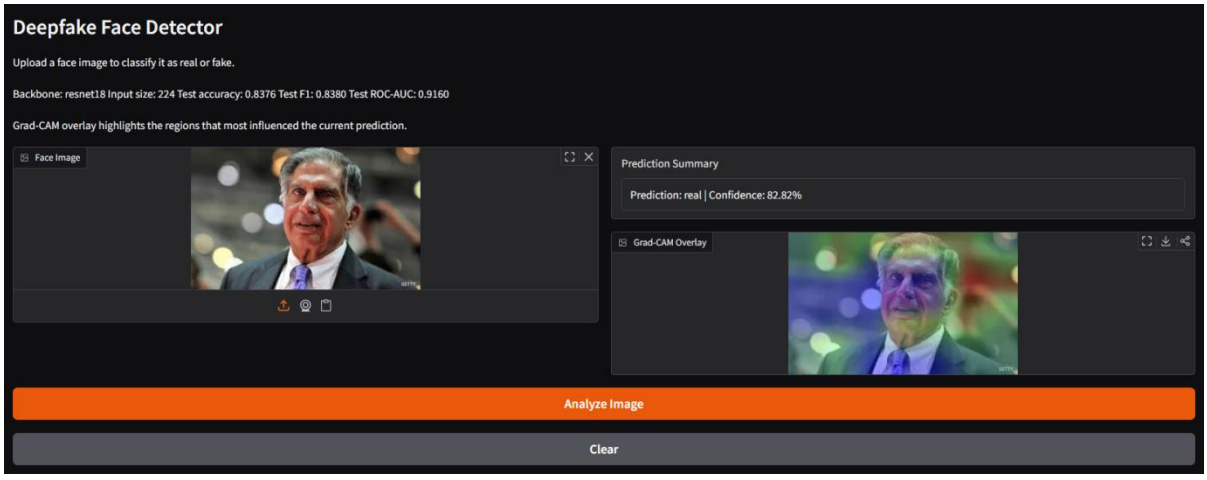
The main execution path of app.py was tested end to end. The parse_args function correctly reads the --checkpoint argument defaulting to artifacts/best_model.pt. The load_artifacts function successfully loads the checkpoint, reconstructs the model architecture from the saved config dictionary, moves the model to the available device, sets eval mode, constructs the evaluation transform, initialises GradCAM on layer4[-1], and pre-renders all analytics charts. The build_interface function correctly constructs the Gradio Blocks layout with all input, output, and analytics components. The demo.launch call starts the server on 127.0.0.1:7860 and the interface is accessible in the browser within 3 seconds of launch.

3. Snapshots of the Project

For Fake Image:



For Real Image:



4. Conclusions

This project successfully implements a complete deepfake face detection system using transfer learning on ResNet18. The system achieves a test accuracy of 83.75%, F1-score of 0.83801, and ROC-AUC of 0.91600 on the 140k Real and Fake Faces dataset. These results represent realistic and balanced generalisation performance — the model correctly classifies 83.47% of fake images and 84.04% of real images with no significant bias toward either class.

The integration of Grad-CAM explainability provides transparency into the model's decision-making process, allowing visual verification that the model focuses on relevant facial regions rather than background artifacts. This is a critical requirement for any practical deepfake detection system where trust and interpretability matter as much as raw accuracy.

The Gradio-based deployment demonstrates that a complete machine learning pipeline — from raw dataset through training, evaluation, explainability, and interactive deployment — can be built entirely on consumer hardware (RTX 4050 Laptop GPU) using open-source tools within a reasonable development timeframe.

An important finding from this project is the relationship between dataset composition and reported accuracy. An initial experiment with a simpler single-layer classification head achieved 99.37% accuracy, which upon analysis was identified as shortcut learning on StyleGAN2-specific texture artifacts rather than genuine deepfake detection capability. The current implementation with a deeper three-layer head achieves more realistic 83.75% accuracy that better reflects the actual difficulty of the classification problem. This critical analysis demonstrates understanding of model evaluation beyond surface-level metrics.

The system has known limitations in domain generalisation — it performs well on the StyleGAN2-based fake faces in the training distribution but would require retraining on more diverse manipulation types (face swaps, face reenactment, diffusion model outputs) for deployment in real-world scenarios.

5. Further Development or Research

- Partial Backbone Unfreezing: Unfreeze the last one or two residual stages of ResNet18 for fine-tuning at a reduced learning rate. This may improve feature adaptation to deepfake-specific artifacts beyond what the frozen backbone can capture.
- Architecture Comparison: Train and compare ResNet18, ResNet34, and ResNet50 on the same dataset and configuration to study the accuracy-complexity tradeoff and determine whether additional depth provides meaningful improvement.
- Dataset Diversity: Collect and train on a more diverse dataset covering multiple deepfake generation techniques — FaceForensics++ video deepfakes, Celeb-DF face swaps, DFDC samples, and diffusion model outputs — to improve cross-domain generalisation.
- Face Detection Preprocessing: Integrate a face detection step (using OpenCV Haar cascades or a pretrained MTCNN) to crop and align detected faces before classification, removing background artifacts and focusing the model exclusively on facial regions.
- Video Deepfake Detection: Extend the system to process video inputs by extracting frames, classifying each frame, and aggregating predictions across frames to produce a video-level verdict.
- Confidence Calibration: Apply temperature scaling or Platt scaling to calibrate the model's confidence scores, ensuring reported confidence values are statistically meaningful and not overconfident.
- Adversarial Robustness: Test the model against adversarial perturbations specifically designed to fool deepfake detectors and explore adversarial training strategies to improve robustness.
- Multi-Model Ensemble: Combine predictions from multiple model architectures trained with different augmentation strategies to reduce individual model bias and improve overall detection reliability.

6. References

[1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of CVPR 2016.

[2] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. Proceedings of ICCV 2017.

[3] Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. Proceedings of ICCV 2019.

[4] Karras, T., Laine, S., & Aila, T. (2019). A Style-Based Generator Architecture for Generative Adversarial Networks. Proceedings of CVPR 2019.

[5] Wang, S. Y., Wang, O., Zhang, R., Owens, A., & Efros, A. A. (2020). CNN-Generated Images Are Surprisingly Easy to Spot... For Now. Proceedings of CVPR 2020.

[6] Li, Y., Yang, X., Sun, P., Qi, H., & Lyu, S. (2020). Celeb-DF: A Large-scale Challenging Dataset for DeepFake Forensics. Proceedings of CVPR 2020.

[7] Loshchilov, I. & Hutter, F. (2019). Decoupled Weight Decay Regularization. Proceedings of ICLR 2019.

[8] Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A., & Ortega-Garcia, J. (2020). Deepfakes and Beyond: A Survey of Face Manipulation and Fake Detection. Information Fusion.

[9] 140k Real and Fake Faces Dataset. Kaggle. https://www.kaggle.com/datasets/xhlulu/140k-real-and-fake-faces

[10] PyTorch Documentation. https://pytorch.org/docs/stable/index.html

[11] Gradio Documentation. https://www.gradio.app/docs/

7. Appendix

Appendix A: Training Configuration

Parameter	Value
Model	resnet18
Image size	224
Batch size	32
Epochs	10
Learning rate	3e-4
Weight decay	1e-4
Num workers	0
Patience (early stopping)	3
Dropout	0.3
Label smoothing	0.05
Seed	42
Freeze backbone epochs	10
Optimiser	AdamW
Scheduler	ReduceLROnPlateau (factor=0.5, patience=2)
Loss function	CrossEntropyLoss with label smoothing 0.05
Mixed precision	torch.amp.autocast + GradScaler (CUDA only)